ZeninFaaS - Trigger-Based Code Dispatching and Load Testing in Kubernetes Environments.

Background:

As more and more companies embrace microservices architectures and tools like Kubernetes to manage containers, it's becoming increasingly clear that managing code execution and handling loads efficiently is crucial. A lot of the traditional solutions out there just don't cut it. They don't seamlessly integrate with Kubernetes, and they might be missing important features like trigger-based dispatching and thorough load testing capabilities. So, there's a real need for solutions that can keep up with the demands of modern containerized environments.

Problem Description:

The issue at hand lies in the shortcomings of current solutions when it comes to smoothly integrating with Kubernetes and providing advanced functionalities such as trigger-based dispatching and thorough load testing. Enter ZENIN FAAS, a solution designed to bridge these gaps. It boasts a user-friendly interface for registering triggers and functions, enabling the seamless dispatching of code or functions based on triggers. Moreover, ZENIN FAAS takes care of managing the pod environment for users and automates tasks like scaling and deleting pods as needed. Additionally, it offers the capability to conduct load testing directly within Kubernetes environments, further enhancing its utility and value.

Design Details and Implementation:

ZeninFaaS comprises three primary components:

- trigger_dispatch: Responsible for code dispatching and load testing functionalities.

- trigger_registry: Facilitates the registration of triggers for code dispatching.

- function_registry: Allows users to register functions to be triggered based on specified conditions.

ZeninFaaS uses the Django web framework and integrates with the Kubernetes Python client for seamless interaction with Kubernetes clusters. The system implements functionalities for trigger registration, function registration, code dispatching, and load testing, providing a

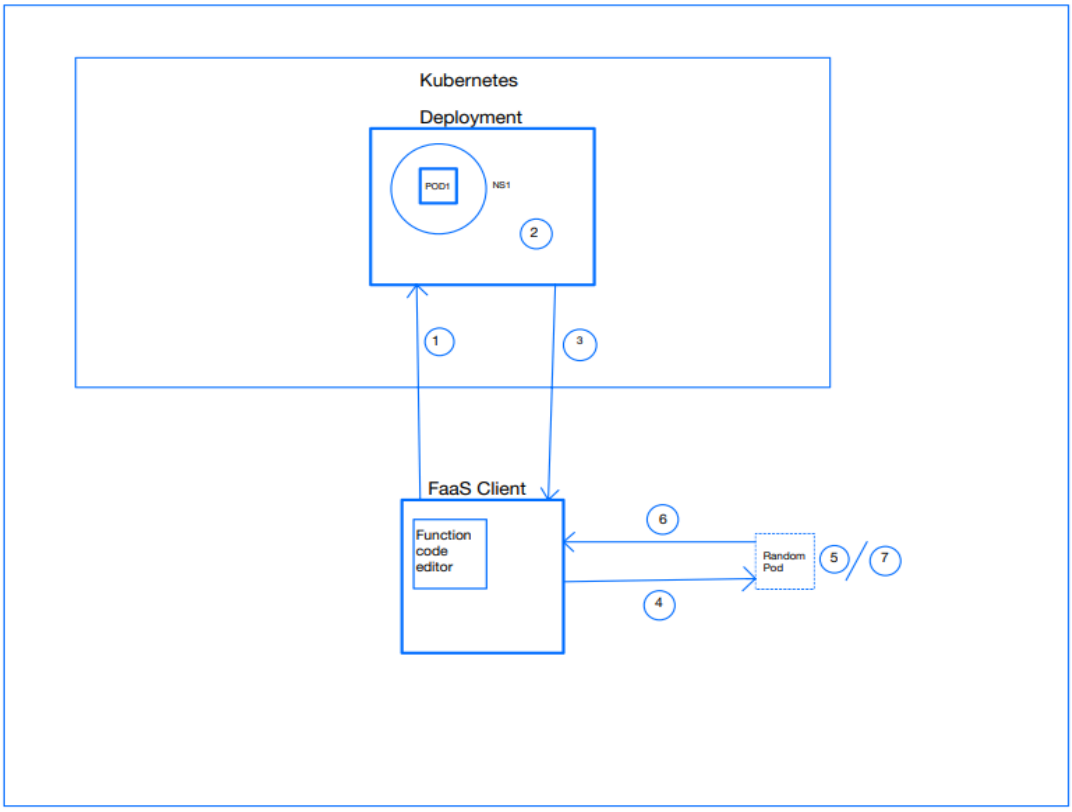
comprehensive solution for managing code execution within Kubernetes environments. ZeninFaaS consists of three versions with each having their pros and cons.

Version 1 (Customizable environment and Manual pod Destruction):

Here each users are assigned a pod to him where he can customize the environment of the pod using a simple interface and run his functions in it by dispatching triggers which run the code in his pod..Users can destroy the resource whenever needed .This is more like amazon EC2 but in a simpler way.

Version 2 (Automatic Pod Creation and Destruction):

Initially there were no pods in Kubernetes server. For each user, a different namespace is assigned and in that namespace a pod will be created dynamically on trigger dispatch. On that respective pod the function code will be executed, and a response will be generated and sent back to the user, and then the pod will get destroyed.Below is architecture used:

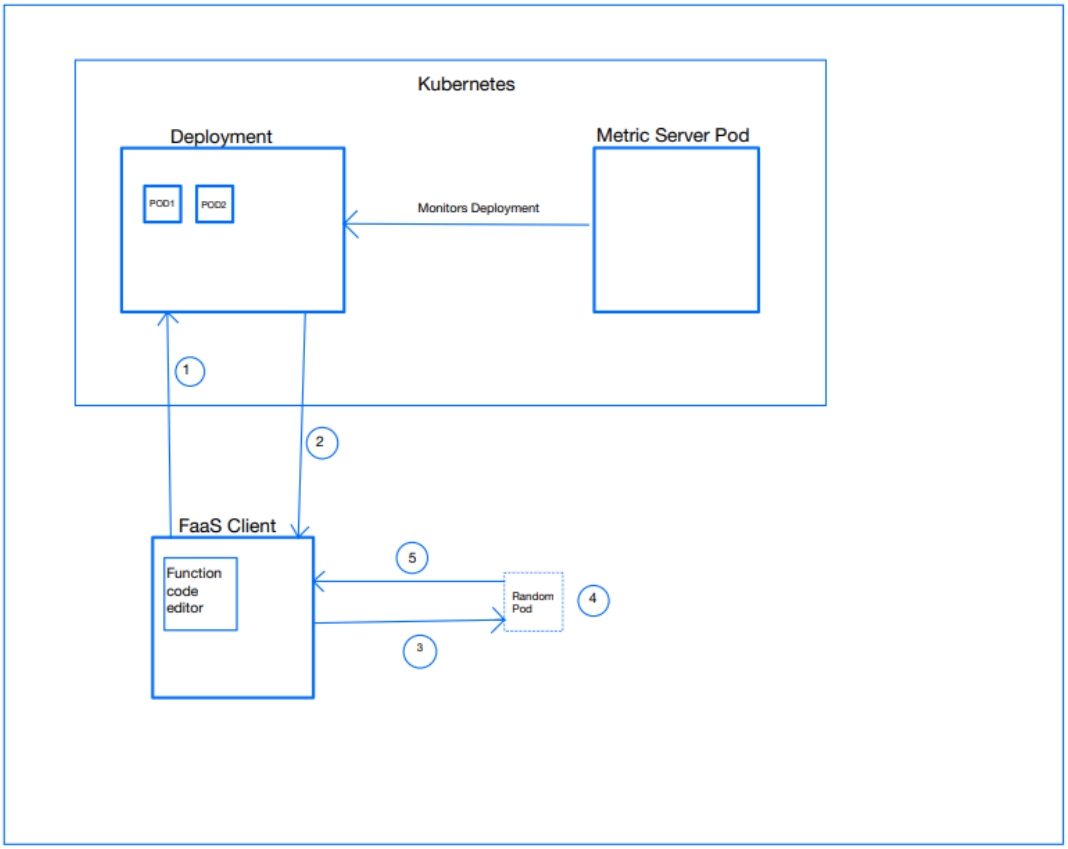


Steps/Workflow -

1) Client dispatches the trigger, and the request is sent to the Kubernetes server.
2) Kubernetes server verifies whether the namespace of the client exists. If namespace does not exist it creates a namespace using its username and, in that namespace, it creates a pod.
3) Kubernetes server returns reference of that pod to the client.
4) The dispatched code is sent to the pod.
5) The pod executes the program.
6) Response of the executed program is returned to the client.
7) Pod is destroyed.

Version 3 (Auto Scaling pods for Dynamic Workloads):

Initially the environment consists of two pods inside a deployment. There is a metric server pod which constantly monitors the deployment, and it collects the pod metrics and based on these metrics it takes the decision to scale the deployment pods.



Steps/Workflow -

1) Client dispatches the trigger, and the request is sent to the Kubernetes server's deployment service which is responsible for assignment of random pods to the client.
2) Deployment sends a random pod to the client.
3) Client dispatches code to its own directory that is created inside the acquired pod.
4) Program is executed and a response is generated.
5) Response is sent to the client.

Difference in Version 2 and Version 3:

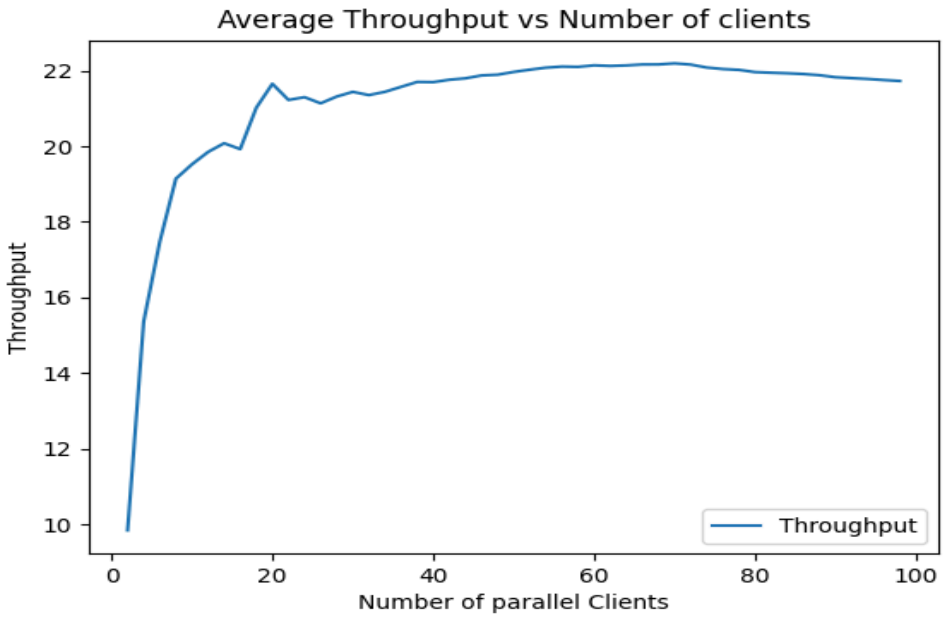
1) Version 2 has proper isolation of pods i.e.; pods are not reused. Once the pod executes the program and sends the response it is deleted whereas in Version 3, we have a certain number of pods readily available and a random pod will be selected and assigned to the client.
2) The time required to get the response of the executed program is higher when compared to Version 3 because in Version 2 we do not have pods readily available, the pods are created dynamically for each request by the client whereas pods in Version 3 are already available to be assigned to the client which is selected randomly.
3) The number of pods in Version 2 is equivalent to the number of requests from the clients whereas in Version 3 the number of pods is directly proportional to the load and is significantly less than the number of requests since already created pods are being reused.
4) There is wastage of CPU resources in Version 2 since a considerable number of pods are being created and destroyed concurrently whereas in Version 3 the resources are efficiently managed by Metric Server Pod.

Experimental Evaluation:

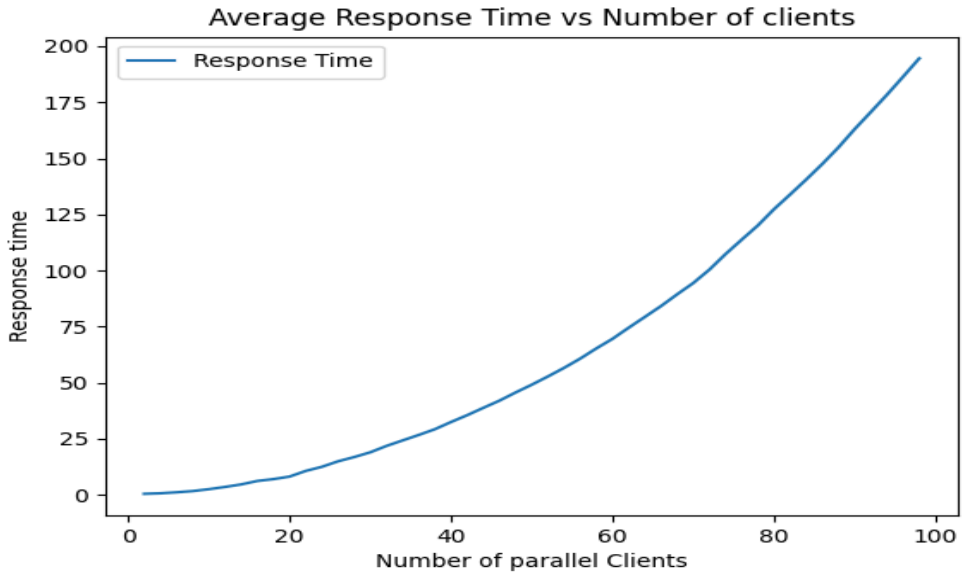
To evaluate the performance of ZENINFAAS, we conducted experiments focusing on response times, throughput, and resource utilization. Various workload scenarios were tested to assess the system's scalability and efficiency in handling different levels of demand. The results provided insights into the system's performance under different conditions, highlighting areas for improvement and optimization.

Version3:(With Autoscaler)

SETUP: 8vcpu ,10 gb ram,100 parallel clients are ran at max with each client sending 2 requests.



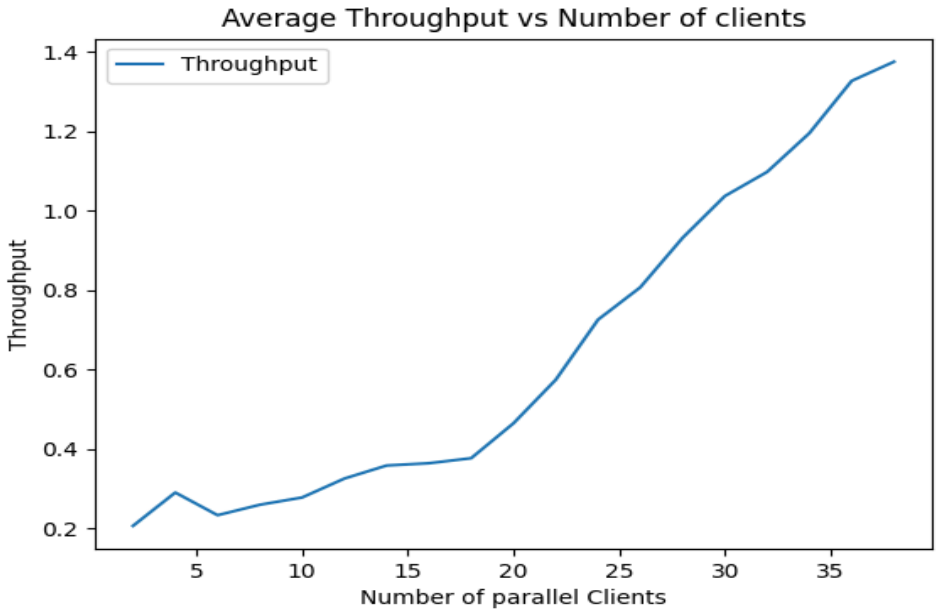
It was observed that the system attends saturation for approx 40 parallel clients where 22 requests are served per second.



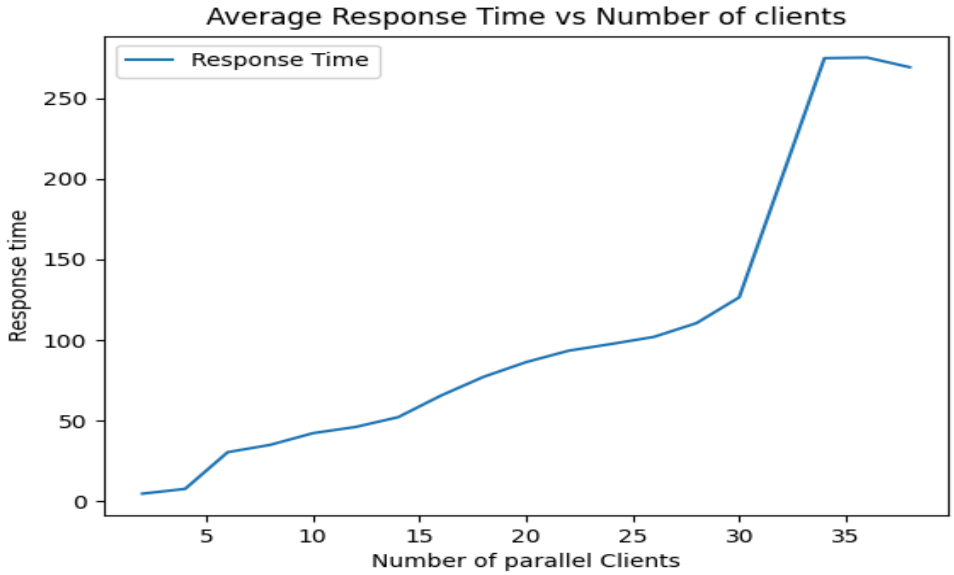
Response time increases as the number of clients increases because of saturation point approx 20 clients are executed per second but our requests are increasing as clients increases.

Version2:(Automatic creation and destruction of pods)

SETUP: 4 vcpu ,4 gb ram,40 parallel clients are ran at max with each client sending 1 requests.



Approx time taken to run a function in this approach was around 4-5 seconds.Here it was observed that for 30 parallel clients around 1 requests are served every 1 sec.



Here for 10 parallel requests the time taken to server all those requests was approximately 45 seconds because for 1 request it takes around 4-5 seconds.

Pros and Cons observed in version2 and 3.

1) Average response time taken in version3 for 40 clients is approx 25 sec where as in version 2 its around 270 sec.Here version 3 is much faster than version2.
2) Time required to server 1 request in version3 is approx -0.3-0.5 sec where as in version 2 its around 4-5sec.Here version 3 is much faster than version2.
3) Main advantage of version2 over version 3 is dynamic pod creation,deletion ,no need to keep some pod running in order to avoid cold start problems which is done by version 3.
4) Version 2 suits best to provide better isolation since a single pod is never shared between users but in version3 pods are shared .

Conclusion

ZENINFAAS serves as a valuable tool for managing code dispatching and conducting load testing in Kubernetes environments. With its user-friendly interface and comprehensive functionalities, ZENINFAAS offers an effective solution for organizations seeking to streamline their code execution processes within Kubernetes clusters. For further details and implementation specifics, refer to the github link :https://github.com/irshadkhan2019/ZeninFaas